



LAB REPORT- 03

Department of Computer Science & Engineering (CSE)
Faculty of Science and Information Technology (FSIT)

Summer 2026

Course Title : Computer Graphics Lab
Course Code : CSE412
Course Credit : 1.5
Date of Submission : July 07 , 2026
Topic Covered : Implementation of Bresenham's
Line Drawing Algorithm Using OpenGL

Submitted By

Shafin Ahmed

232-15-184

65_M1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

LAB REPORT 03

Implementation of Bresenham's Line Drawing Algorithm Using OpenGL

1. Introduction

Computer Graphics is an important branch of computer science that focuses on generating and manipulating visual objects using computational techniques. One of the fundamental operations in computer graphics is line drawing, which is used extensively in designing graphical objects, user interfaces, games, simulations, and computer-aided design systems.

Various line drawing algorithms have been developed to efficiently generate straight lines on raster display devices. Among them, Bresenham's Line Drawing Algorithm is considered one of the most efficient because it utilizes integer arithmetic rather than floating-point calculations, resulting in faster execution and better performance.

In this laboratory experiment, I implemented Bresenham's Line Drawing Algorithm using OpenGL to design a 2D house. The program was designed to automatically determine whether the slope of a line belongs to the range:

- $0 \leq m \leq 1$
- $m > 1$

and then execute the corresponding Bresenham logic automatically.

Before starting the implementation, I configured the OpenGL development environment using Code::Blocks IDE with GLUT libraries. I also used Desmos Graphing Calculator to determine the approximate coordinates of the house structures before implementing them in OpenGL.

This experiment helped me understand raster graphics systems, line generation algorithms, decision parameters, coordinate systems, and graphical object construction techniques.

Environment / Platform Setup

Component	Description
Operating System	Windows 10
IDE	Code::Blocks
Programming Language	C
Graphics Library	OpenGL
Utility Toolkit	GLUT
Coordinate System	0 – 100
Window Size	850 × 725
Graph Tool	Desmos

2. Problem Statement

The objective of this laboratory experiment was to design and implement a colorful 2D house using Bresenham's Line Drawing Algorithm in OpenGL.

The program was required to automatically detect the slope of every line segment and determine whether:

- $0 \leq m \leq 1$
- $m > 1$

and execute the appropriate Bresenham algorithm automatically.

The project requirements included:

- Designing two 2D houses side by side.
- Implementing automatic slope detection.
- Changing the coordinate system to 0–100.
- Setting the OpenGL window size to 850×725 pixels.
- Using RGB color values in the 0–255 format.
- Using at least five different colors.
- Drawing all objects entirely using Bresenham's algorithm.
- Creating thick visible line segments suitable for documentation purposes.

The project consists of several graphical components including:

- House body
- Roof
- Door

- Window
- Chimney

All components were generated using Bresenham's Line Drawing Algorithm without using OpenGL's built-in line drawing functions.

Project Description

In this experiment, I implemented a generalized Bresenham line drawing function that automatically determines the slope category of each line.

The program first calculates:

- Difference in x-coordinate (dx)
- Difference in y-coordinate (dy)

Then it automatically determines whether:

Case 1:

If $dx \geq dy$

the algorithm executes the logic for:

$$0 \leq m \leq 1$$

where the x-coordinate is incremented.

Case 2:

If $dx < dy$

the algorithm executes the logic for:

$$m > 1$$

where the y-coordinate is incremented.

Using this approach, the same function can successfully draw lines with different slopes without requiring separate implementations.

The project contains two realistic houses consisting of:

- Cream-colored house body
- Brick-red roof
- Brown door
- Sky-blue window
- Gray chimney

A white background was used to improve visibility and produce a professional appearance suitable for a laboratory report.

3. Source Code

Full Source Code:

```
// Shafin Ahmed
// ID: 232-15-184
// Section: 65-M1
// Lab Report 03
// 2D House using Bresenham Line Drawing Algorithm

#include <GL/gl.h>
#include <GL/glut.h>
#include <stdlib.h>

void drawBresenham(int x0,int y0,int x1,int y1)
{
    int dx = abs(x1-x0);
    int dy = abs(y1-y0);

    int sx = (x0<x1)?1:-1;
    int sy = (y0<y1)?1:-1;

    glBegin(GL_POINTS);

    glVertex2i(x0,y0);

    // Automatic slope detection
    if(dx >= dy) // 0<=m<=1
    {
        int p = 2*dy-dx;

        while(x0!=x1)
        {
            x0 += sx;

            if(p<0)
                p += 2*dy;
            else
```

```

        {
            y0 += sy;
            p += 2*dy-2*dx;
        }

        glVertex2i(x0,y0);
    }
}
else // m>1
{
    int p = 2*dx-dy;

    while(y0!=y1)
    {
        y0 += sy;

        if(p<0)
            p += 2*dx;
        else
        {
            x0 += sx;
            p += 2*dx-2*dy;
        }

        glVertex2i(x0,y0);
    }
}

glEnd();
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    //-----
    // House Body (Cream)
    //-----
    glColor3f(245.0/255,222.0/255,179.0/255);

    drawBresenham(10,20,40,20);
    drawBresenham(40,20,40,50);
    drawBresenham(40,50,10,50);
    drawBresenham(10,50,10,20);

    //-----
    // Roof (Brick Red)
    //-----
    glColor3f(178.0/255,34.0/255,34.0/255);

    drawBresenham(10,50,25,70);
    drawBresenham(25,70,40,50);

    //-----

```

```

// Door (Brown)
//-----
glColor3f(101.0/255,67.0/255,33.0/255);

drawBresenham(21,20,21,35);
drawBresenham(29,20,29,35);
drawBresenham(21,35,29,35);

//-----
// Window (Sky Blue)
//-----
glColor3f(135.0/255,206.0/255,235.0/255);

drawBresenham(13,35,18,35);
drawBresenham(18,35,18,42);
drawBresenham(18,42,13,42);
drawBresenham(13,42,13,35);

//-----
// Chimney (Gray)
//-----
glColor3f(120.0/255,120.0/255,120.0/255);

drawBresenham(31,60,31,72);
drawBresenham(36,60,36,72);
drawBresenham(31,72,36,72);

//=====
// SECOND HOUSE
//=====

//-----
// House Body
//-----
glColor3f(245.0/255,222.0/255,179.0/255);

drawBresenham(55,20,85,20);
drawBresenham(85,20,85,50);
drawBresenham(85,50,55,50);
drawBresenham(55,50,55,20);

//-----
// Roof
//-----
glColor3f(178.0/255,34.0/255,34.0/255);

drawBresenham(55,50,70,70);
drawBresenham(70,70,85,50);

//-----
// Door
//-----
glColor3f(101.0/255,67.0/255,33.0/255);

```

```

drawBresenham(66,20,66,35);
drawBresenham(74,20,74,35);
drawBresenham(66,35,74,35);

//-----
// Window
//-----
glColor3f(135.0/255,206.0/255,235.0/255);

drawBresenham(58,35,63,35);
drawBresenham(63,35,63,42);
drawBresenham(63,42,58,42);
drawBresenham(58,42,58,35);

//-----
// Chimney
//-----
glColor3f(120.0/255,120.0/255,120.0/255);

drawBresenham(76,60,76,72);
drawBresenham(81,60,81,72);
drawBresenham(76,72,81,72);

glFlush();
}

void init()
{
    // White background
    glClearColor(1.0,1.0,1.0,1.0);

    // Thick pixels for visibility
    glPointSize(8.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Coordinate system
    glOrtho(0,100,0,100,-1,1);
}

int main(int argc,char **argv)
{
    glutInit(&argc,argv);

    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);

    // Window size
    glutInitWindowSize(850,725);

    glutInitWindowPosition(100,50);

    glutCreateWindow("2D House using Bresenham Algorithm by Shafin Ahmed");

```



```
init();  
glutDisplayFunc(display);  
glutMainLoop();  
return 0;  
}
```

Code Description

The program was developed using modular programming concepts.

Step 1: OpenGL Initialization

The OpenGL environment was initialized using GLUT functions. The coordinate system was configured from 0 to 100 using the orthographic projection.

Step 2: Implementation of Bresenham Function

A generalized Bresenham function named: **drawBresenham()** was developed.

This function automatically determines whether:

- $dx \geq dy$
- $dx < dy$

and applies the appropriate Bresenham decision parameter formula.

Step 3: Drawing the House Structure

The first house was created using multiple Bresenham line segments:

- House boundary
 - Roof
 - Door
 - Window
 - Chimney
-

Step 4: Drawing the Second House

A second house was created using similar coordinates shifted toward the right side of the screen.

Step 5: Color Management

The colors were specified using RGB values ranging from 0 to 255 and converted into OpenGL normalized values by dividing each component by 255.

Examples:

1. 245/255
 2. 178/255
 3. 101/255
-

Step 6: Visibility Enhancement

To improve visibility in the lab report screenshots, the following modifications were applied:

- White background
 - Thick point size
 - Realistic house color combinations
 - Large window size
-

4. Output

The program successfully generated two colorful houses using Bresenham's Line Drawing Algorithm.

The generated output satisfied all requirements:

- ✓ Automatic slope detection
- ✓ Coordinate system 0–100
- ✓ Window size 850×725
- ✓ RGB color conversion
- ✓ Five different colors
- ✓ Thick line visibility
- ✓ Two houses

Screenshot of Source Code

```
main.cpp x
1 // Shafin Ahmed
2 // ID: 232-15-184
3 // Section: 65-M1
4 // Lab Report 03
5 // 2D House using Bresenham Line Drawing Algorithm
6
7 #include <GL/gl.h>
8 #include <GL/glut.h>
9 #include <stdlib.h>
10
11 void drawBresenham(int x0,int y0,int x1,int y1)
12 {
13     int dx = abs(x1-x0);
14     int dy = abs(y1-y0);
15
16     int sx = (x0<x1)?1:-1;
17     int sy = (y0<y1)?1:-1;
18
19     glBegin(GL_POINTS);
20
21     glVertex2i(x0,y0);
22
23     // Automatic slope detection
24     if(dx >= dy) // 0<=m<=1
25     {
```

```
main.cpp x
25     {
26         int p = 2*dy-dx;
27
28         while(x0!=x1)
29         {
30             x0 += sx;
31
32             if(p<0)
33                 p += 2*dy;
34             else
35             {
36                 y0 += sy;
37                 p += 2*dy-2*dx;
38             }
39             glVertex2i(x0,y0);
40         }
41     }
42
43     else // m>1
44     {
45         int p = 2*dx-dy;
46
47         while(y0!=y1)
48         {
49             y0 += sy;
```

```
main.cpp x
49             y0 += sy;
50
51             if(p<0)
52                 p += 2*dx;
53             else
54             {
55                 x0 += sx;
56                 p += 2*dx-2*dy;
57             }
58             glVertex2i(x0,y0);
59         }
60     }
61 }
62
63 glEnd();
64 }
65
66 void display()
67 {
68     glClear(GL_COLOR_BUFFER_BIT);
69
70     //-----
71     // House Body (Cream)
72     //-----
```

```

main.cpp x
73     glColor3f(245.0/255,222.0/255,179.0/255);
74
75     drawBresenham(10,20,40,20);
76     drawBresenham(40,20,40,50);
77     drawBresenham(40,50,10,50);
78     drawBresenham(10,50,10,20);
79
80     //-----
81     // Roof (Brick Red)
82     //-----
83     glColor3f(178.0/255,34.0/255,34.0/255);
84
85     drawBresenham(10,50,25,70);
86     drawBresenham(25,70,40,50);
87
88     //-----
89     // Door (Brown)
90     //-----
91     glColor3f(101.0/255,67.0/255,33.0/255);
92
93     drawBresenham(21,20,21,35);
94     drawBresenham(29,20,29,35);
95     drawBresenham(21,35,29,35);
96
97     //-----

```

```

main.cpp x
97     //-----
98     // Window (Sky Blue)
99     //-----
100    glColor3f(135.0/255,206.0/255,235.0/255);
101
102    drawBresenham(13,35,18,35);
103    drawBresenham(18,35,18,42);
104    drawBresenham(18,42,13,42);
105    drawBresenham(13,42,13,35);
106
107    //-----
108    // Chimney (Gray)
109    //-----
110    glColor3f(120.0/255,120.0/255,120.0/255);
111
112    drawBresenham(31,60,31,72);
113    drawBresenham(36,60,36,72);
114    drawBresenham(31,72,36,72);
115
116    //=====
117    // SECOND HOUSE
118    //=====
119
120    //-----
121    // House Body

```

```

main.cpp x
121    // House Body
122    //-----
123    glColor3f(245.0/255,222.0/255,179.0/255);
124
125    drawBresenham(55,20,85,20);
126    drawBresenham(85,20,85,50);
127    drawBresenham(85,50,55,50);
128    drawBresenham(55,50,55,20);
129
130    //-----
131    // Roof
132    //-----
133    glColor3f(178.0/255,34.0/255,34.0/255);
134
135    drawBresenham(55,50,70,70);
136    drawBresenham(70,70,85,50);
137    void drawBresenham(int x0, int y0, int x1, int y1)
138    //-----
139    // Door
140    //-----
141    glColor3f(101.0/255,67.0/255,33.0/255);
142
143    drawBresenham(66,20,66,35);
144    drawBresenham(74,20,74,35);
145    drawBresenham(66,35,74,35);

```

```

main.cpp x
145     drawBresenham(66,35,74,35);
146
147     //-----
148     // Window
149     //-----
150     glColor3f(135.0/255,206.0/255,235.0/255);
151
152     drawBresenham(58,35,63,35);
153     drawBresenham(63,35,63,42);
154     drawBresenham(63,42,58,42);
155     drawBresenham(58,42,58,35);
156
157     //-----
158     // Chimney
159     //-----
160     glColor3f(120.0/255,120.0/255,120.0/255);
161
162     drawBresenham(76,60,76,72);
163     drawBresenham(81,60,81,72);
164     drawBresenham(76,72,81,72);
165
166     glFlush();
167 }
168
169 void init()
...

main.cpp x
169 void init()
170 {
171     // White background
172     glClearColor(1.0,1.0,1.0,1.0);
173
174     // Thick pixels for visibility
175     glPointSize(8.0);
176
177     glMatrixMode(GL_PROJECTION);
178     glLoadIdentity();
179
180     // Coordinate system
181     glOrtho(0,100,0,100,-1,1);
182 }
183
184 int main(int argc, char **argv)
185 {
186     glutInit(&argc,argv);
187
188     glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
189
190     // Window size
191     glutInitWindowSize(850,725);
192
193     glutInitWindowPosition(100,50);
...

193     glutInitWindowPosition(100,50);
194
195     glutCreateWindow("2D House using Bresenham Algorithm by Shafin Ahmed");
196
197     init();
198
199     glutDisplayFunc(display);
200
201     glutMainLoop();
202
203     return 0;
204 }
205

```

Figure 1: Source Code in Code::Blocks

Screenshot of Program Output

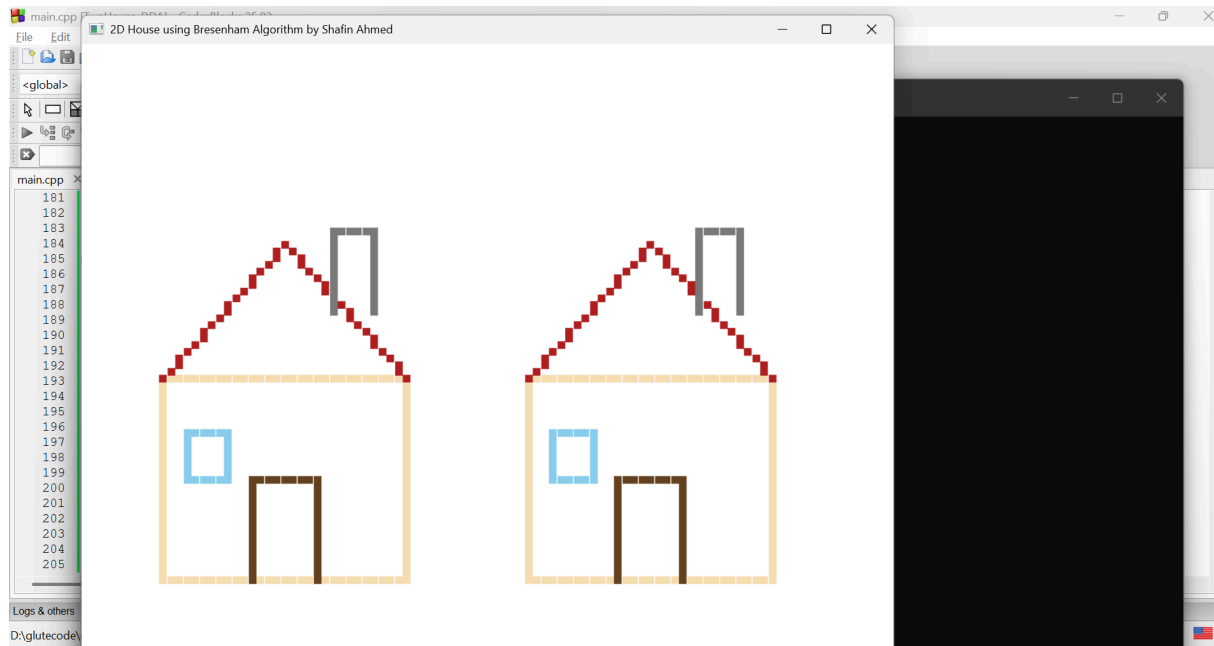


Figure 2: Output of Two Houses Using Bresenham Algorithm

5. Graph

Before implementing the project in OpenGL, I used the Desmos Graphing Calculator to plot the coordinates of the houses.

Using Desmos allowed me to:

- Verify the coordinate positions.
- Visualize the geometric structure.
- Reduce implementation errors.
- Ensure symmetry of the houses.

The graph contained:

- House boundary coordinates
- Roof coordinates
- Door coordinates
- Window coordinates
- Chimney coordinates

Using Desmos significantly simplified the implementation process and improved the accuracy of the graphical design.

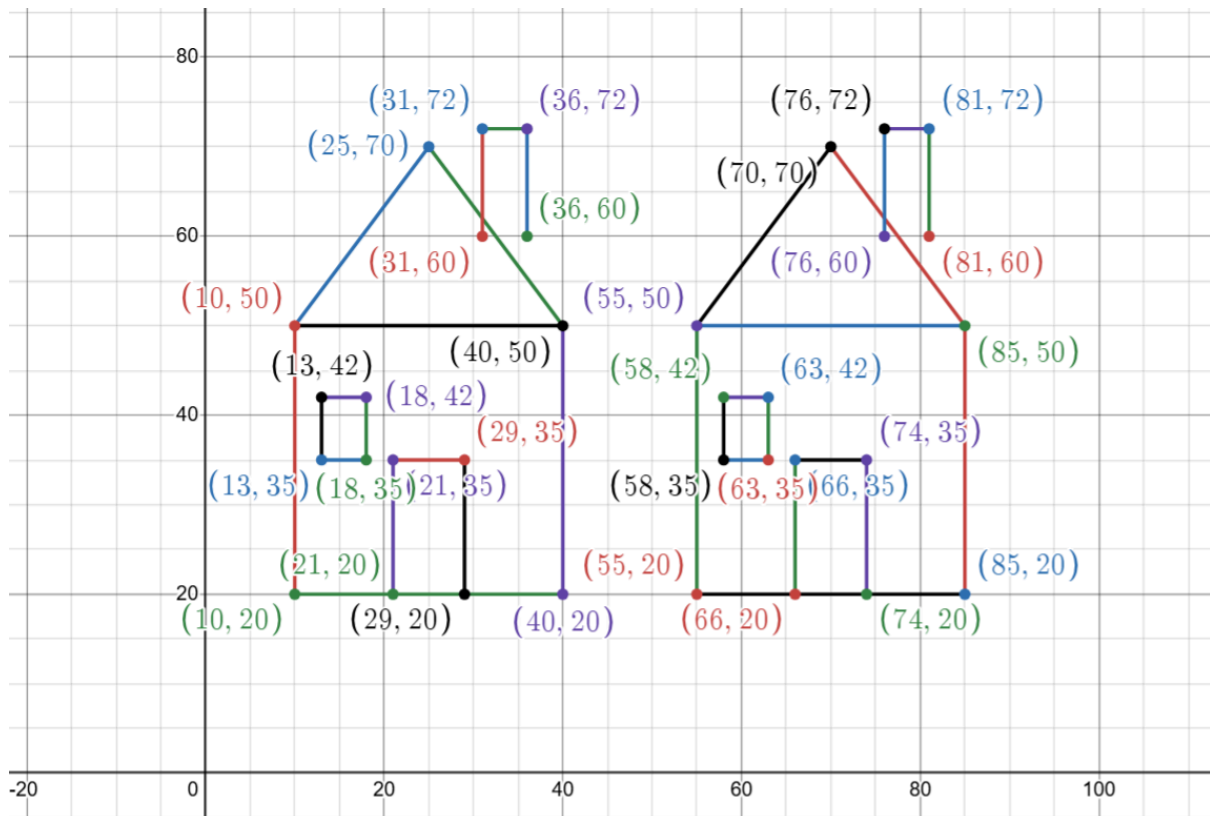


Figure 3: Desmos Graph of the House Coordinates

6. Discussion

This laboratory experiment provided practical knowledge about raster graphics systems and Bresenham's Line Drawing Algorithm.

Unlike the DDA algorithm, Bresenham's algorithm uses integer arithmetic operations, which makes it significantly faster and more efficient.

During the implementation, I learned:

- Raster graphics concepts
- Decision parameter calculations
- Coordinate system transformations
- OpenGL rendering
- Color management
- Geometric object construction
- Automatic slope detection techniques

The implementation of a generalized Bresenham function helped me understand how different slope conditions can be handled using a single algorithm.

Using Desmos before coding also improved my understanding of coordinate geometry and graphical planning.

Questions to Discuss

1. Why is Bresenham faster than DDA?

Bresenham's algorithm is faster than DDA because it uses only integer arithmetic operations instead of floating-point calculations. Integer operations require less processing time and make the algorithm highly efficient for raster graphics systems.

2. What is the purpose of the decision parameter?

The decision parameter determines which pixel should be selected during each iteration of the algorithm. It helps decide whether the next pixel should move horizontally, vertically, or diagonally to approximate the ideal line accurately.

3. Why does Bresenham avoid floating-point arithmetic?

Floating-point operations are computationally expensive and may introduce rounding errors. Bresenham uses integer arithmetic to improve efficiency and accuracy.

4. What are the two candidate pixels considered at each step?

Depending on the slope, the algorithm considers:

- East (E)
- North-East (NE)

or

- North (N)

- North-East (NE)

The decision parameter selects the appropriate pixel.

5. Why is Bresenham preferred in raster graphics systems?

Bresenham's algorithm is preferred because it:

- Uses integer arithmetic
 - Executes quickly
 - Produces accurate results
 - Requires low memory
 - Is suitable for real-time applications
-

6. What limitations does this version of the algorithm have?

The limitations include:

- It only draws straight lines.
 - Curves require separate algorithms.
 - Anti-aliasing is not supported.
 - Complex shapes require many line segments.
 - Additional logic may be needed for advanced graphical applications.
-

Conclusion

In this laboratory experiment, I successfully implemented Bresenham's Line Drawing Algorithm using OpenGL to design two realistic 2D houses. The program automatically detected the slope of each line segment and selected the appropriate Bresenham logic accordingly.

This experiment enhanced my understanding of raster graphics systems, line drawing algorithms, coordinate systems, graphical object construction, and OpenGL programming. I also learned the importance of algorithm optimization and geometric planning through the use of Desmos.

Overall, this experiment provided valuable practical experience that can be applied in computer graphics, game development, computer-aided design, graphical user interfaces, and visualization systems.
